

Interaction Combinators for Grid Computing: Experimental Evaluation of Distributed Reduction with Strong Confluence

Filipe Andrade Nascimento
Yuri Faro Dantas de Sant Anna (Advisor)
Universidade Tiradentes

Abstract—Grid computing aims to distribute work over wide-area networks, where latency varies and parts of the system may fail without warning. In this setting, a model of computation whose result is deterministic by construction is a natural candidate. This work investigates whether Lafont’s interaction combinators (IC), which compute by local graph rewriting, can play that role. Their central property is strong confluence: the rules, applied in any order, lead to the same final net. Prior work explored this property only in shared memory, not on a grid. The central hypothesis of this work is that strong confluence, combined with a protocol that preserves the net’s structure, makes the distributed reduction of a terminating net converge to the same normal form as the sequential reduction, for any number of workers. Relativist was built to test this hypothesis. The proposed model is a distributed reducer, implemented in Rust and developed under *Spec-Driven Development*, with 28 formal specifications and about 800 traceable requirements. Three campaigns exercised it: local execution, TCP communication over *localhost* via containers, and a sweep spanning four orders of magnitude in net size. Across about 5350 reductions, the distributed normal form matched the sequential one in every case; a break-even analysis indicates the bandwidth and per-interaction cost at which distribution becomes advantageous. To date, no other empirical validation of this equivalence on a grid architecture has been found. The result supports the use of models with structural determinism in distributed systems subject to failures.

Index Terms—interaction combinators, grid computing, strong confluence, distributed reduction, graph rewriting

1 INTRODUCTION

GRID computing environments coordinate computational resources spread across virtual organizations that form and dissolve dynamically, in the classic definition of Foster, Kesselman, and Tuecke [1]. When they communicate over wide-area networks, these resources may exhibit high and variable latency, message loss and delay, and no central control [2]. In these settings, resources execute processes in unpredictable orders, and so keeping state consistent requires explicit coordination. This scenario motivates a model of computation whose result is deterministic by construction, without global coordination.

Interaction combinators (IC) [3] are a minimal model of computation: three symbols and six rules that rewrite a graph. Two properties matter for this work. The first is *strong confluence*: the rules, applied in any order, produce the same final result. The second is *locality*: each computation step affects only the two agents that interact. Despite its simplicity, the system is *universal*; Lafont [3] proves that any computation expressible in *interaction nets* [4] can be translated into it. Section 2.1 formalizes these concepts.

To date, the works that set out to explore these properties did so in shared memory. HVM2 [5] is a massively parallel evaluator based on extended IC. It achieved a speedup nearly proportional to the number of cores. However, the system depends on shared memory. Mackie [6] proposed, in 1997, partitioning interaction nets across distributed-memory processors, but the proposal was not evaluated in a grid setting. The properties that underpin parallelism in shared memory thus remain untested in grid computing.

The objective of this work is to evaluate experimentally whether the strong confluence and locality of IC, supported by a protocol that preserves the net’s structure, suffice to build a distributed reduction on a grid whose result coincides with that of the sequential reduction, for any number of workers, provided the net terminates. The hypothesis is affirmative, for two reasons pointed out by the literature. HVM2 [5] showed that these properties support real parallelism in shared memory. And strong confluence, being independent of the order and location at which each active pair is reduced, presupposes neither shared memory nor a specific physical topology. This work tests the hypothesis empirically.

More precisely, the hypothesis is that strong confluence, combined with four guarantees of the protocol, leads to the same normal form as the sequential reduction, for any number of workers and any work order. The four guarantees are to partition without losing connections, to re-compose boundaries correctly, to assign identifiers without collision, and to terminate in a finite number of rounds. Section 4 states these premises formally. To test it, Relativist was built¹, a system in Rust developed under *Spec-Driven Development* (28 specifications and about 800 formal requirements). On each run, the system checks whether $\text{reduce_all}(\text{net}) \cong \text{run_grid}(\text{net}, n)$, that is, whether the two

1. Relativist: an open-source distributed IC reducer in Rust. Release v0.20.0-pre, available at <https://github.com/andrade-filipe/relativist>. The experimental campaigns and their provenance manifests (with SHA-256 checksums) accompany the repository.

reductions produce isomorphic graphs. The contribution of this work is to demonstrate that equivalence with experimental data.

The remainder of the paper is organized as follows. Section 2 presents the theoretical background. Section 3 discusses related work. Section 4 describes the methodology. Section 5 presents the results and discussion. Section 6 concludes the paper and points to future work.

2 THEORETICAL BACKGROUND

This section addresses only the formal model of IC and the properties that make it a candidate for distributed reduction. The prior implementations and attempts to explore it, in shared or distributed memory, are analyzed in Section 3.

2.1 The Formal Model: from Interaction Nets to Interaction Combinators

Lafont [4] proposed *interaction nets* in 1990. It is a model of computation based on graph rewriting. The graph’s vertices are the *agents*, each labeled and representing an operation. Every agent has a fixed number of *ports*. One of them, the *principal port*, is the only one through which the agent interacts; the rest are *auxiliary ports*, through which data enters and leaves. Two agents connected by their principal ports form an *active pair* (*redex*). This pair is the smallest unit of computation, equivalent to an instruction ready to execute. To reduce is to apply to the pair the corresponding rewrite rule, which replaces it with a new configuration of agents and connections.

Three restrictions on the rewrite rules make the model suitable for distributed computation. The first, *linearity*, requires each variable to appear exactly once on each side of the rule, so that nothing is implicitly duplicated or discarded. The second, *binary interaction*, allows interaction only between two agents linked by their principal ports. The third, *absence of ambiguity*, admits at most one rule for each pair of symbols. In the original 1990 version [4], the third condition forbade a symbol from interacting with itself. Lafont removed that prohibition in 1997 [3, note 1], allowing the *gamma-gamma*, *delta-delta*, and *epsilon-epsilon* interactions, which are the basis of IC.

Together, these three conditions guarantee *strong confluence*. If a net N can be reduced in two different ways, both lead to the same result. Formally, if N reduces in one step to both P and Q , with $P \neq Q$, then both reduce, also in one step, to the same net R [4, Proposition 1]. Thus the active pairs are independent of one another and can be reduced in any order, by any processor, without mutual interference [4]. *Locality* reinforces this independence, since each interaction affects only the two agents of the pair and their immediate ports, without altering distant points of the net. Together, the two properties allow, in principle, reducing all active pairs simultaneously. Lafont also proposed a type discipline that avoids *deadlock* in well-typed nets [4, Propositions 5–7], an aspect this paper does not develop.

In 1997, Lafont [3] reduced the model to the minimum. IC are a particular case of interaction nets with only three symbols and six rules (Fig. 1). The symbols are *gamma* and *delta*, both with two auxiliary ports, and *epsilon*, with none.

Half of the rules deal with equal symbols and are called *annihilation*. The *gamma-gamma* rule reconnects the auxiliary ports crosswise (port 1 of one goes to port 2 of the other), *delta-delta* reconnects them in parallel (port 1 with port 1, port 2 with port 2), and *epsilon-epsilon* erases both agents. The other half deals with different symbols and is called *commutation*. The *gamma-delta* rule replaces the pair with four new agents reconnected to the original ports; *gamma-epsilon* and *delta-epsilon* erase the two-port agent and leave two *epsilon*s in its place.

The two annihilations are not the same. It is *gamma-gamma*, and not *delta-delta*, that swaps the auxiliary ports [3, p. 82]. The distinction matters, because Lafont shows that both rules are necessary for the system’s universality [3, p. 90]. The central result is that any system of interaction nets can be translated into these three combinators without losing complexity or parallelism (Theorem 1 and Proposition 4 of [3]). Computation proceeds while an active pair exists; when none remains, the net has reached its *normal form*, which is the result of the computation.

Lafont also showed that removing any of the rules (except *delta-epsilon*) breaks universality, and conjectured that there is no simpler system: “we conjecture that our system of interaction combinators is essentially the simplest one satisfying Theorem 1” [3, p. 90]. Mackie [6] observed that these combinators play, for interaction nets, a role analogous to that of the S and K combinators in combinatory logic.

Later works consolidated the theory of IC. Mackie and Pinto [7] showed that the main fragments of linear logic, namely the multiplicative, the exponential, and the additive, can be encoded in them, although not all cases are covered. Mazza and Ross [8] analyzed the symmetric variant of the system (of the same expressive power) and proved that strong confluence induces a precise notion of semantic equivalence; with that, they reaffirmed interaction nets as “a model of distributed and deterministic computation” [8, p. 316]. Lafont summarizes: computation time is “relativistic”, since it dispenses with a global clock [3].

3 RELATED WORK

This section examines the prior implementations and attempts to explore the properties of IC, from shared to distributed memory, and what each one revealed about distributing reduction.

HVM2 [5] is the main empirical evidence that the properties of IC enable real parallelism. It is a massively parallel evaluator that compiles high-level programs to C and CUDA, using extended IC (with additional agents REF, NUM, OPE, and SWI for practical performance) as an intermediate representation. Each agent occupies 64 bits, allowing atomic interactions without *locks*. On CPUs, the system uses *task-stealing* (dynamic redistribution of active pairs among threads); on GPUs, sharing annotations avoid divergence among threads of the same *warp* (a group of 32 NVIDIA threads that execute the same instruction simultaneously). Preliminary results reported by Taelin [5] indicate up to 74000 MIPS on an NVIDIA RTX 4090 with 32768 threads and near-linear speedup. These figures come from a preprint whose detailed benchmark section has not yet been published; they should be treated as indicative. HVM2, like

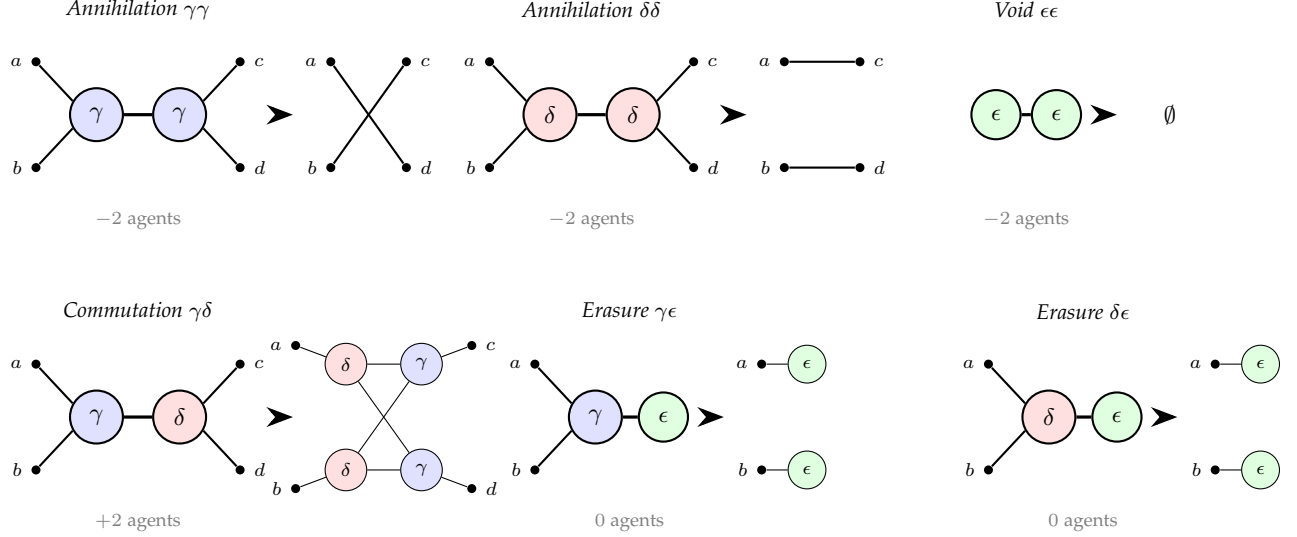


Figure 1: The six interaction rules of IC. Terminals a, b, c, d indicate preserved external connections; the thick line indicates the connection between principal ports (active pair). Top row, annihilations: $\gamma\gamma$ reconnects crosswise ($a-d, b-c$), $\delta\delta$ reconnects in parallel ($a-c, b-d$), $\epsilon\epsilon$ erases both. Bottom row: commutation $\gamma\delta$ (+2 agents, bipartite net $K_{2,2}$) and erasures $\gamma\epsilon, \delta\epsilon$ (0 agents). Both binary annihilations are necessary for universality [3]. Colors: blue = γ , red = δ , green = ϵ . Diagram produced by the author in TikZ with AI assistance.

the present work, operates on terminating programs. This restriction is formalized as premise P6 in Section 4.

However, HVM2 operates exclusively in shared memory, where all threads access a single buffer of agents. In distributed memory, where workers communicate over a network, questions arise that locality alone does not resolve: how to partition the net across processors? How to resolve active pairs whose agents reside on different resources? Strong confluence guarantees determinism of the result, but says nothing about the cost of a poorly made distribution.

Mackie [6] proposed a static-analysis technique, grounded in abstract interpretation, to partition interaction nets across processors. Two lessons follow from it: net partitioning strongly affects performance, and it is possible to predict statically, approximately, which agents will interact (the general case is undecidable). At the time, the author himself admitted that no “successful distributed or parallel implementation” [6] existed. In shared memory, Kahl [9] obtained speedups of up to $3.8\times$ on 9 cores with a parallel implementation in Haskell, but did not extend the work to distributed memory.

On the formal side, Pinto [10] built the first abstract machine for reducing interaction nets, decomposing each interaction into atomic operations over a configuration of *Heap*, *Stack*, and *Involutions*, and proved that both the sequential and the concurrent version (with POSIX threads) are correct. The lesson relevant to this work is that strong confluence suffices to guarantee concurrent correctness, provided it is accompanied by an adequate synchronization protocol. This is the principle of the BSP protocol of this work (Section 4). Pinto’s machine, however, is restricted to shared memory. Mackie and Sato [11] investigated which interaction-net shapes benefit from parallel evaluation. They separate two kinds of operation: *batch*, which only deliver a result after consuming all the data, and *streaming*, which return partial

results and allow more parallelism. The conclusion is that the shape of the workload determines how much parallelism can be extracted. In Fibonacci with *streaming* addition, for example, parallel complexity drops from exponential to quadratic. This idea guides the three benchmark profiles of Section 4: Profile A gathers pure batch workloads, Profile B mixes batch and streaming, and Profile C covers the worst case.

At the opposite end is grid computing without a formal guarantee of confluence. Wang et al. [12] recorded the engineering cost of that absence when describing DDGrid, a grid for drug discovery. The system required a SEDA architecture (*Staged Event-Driven Architecture*), failure detection by complex event processing, backup execution for the slowest workers, and managing more than 1 000 concurrent tasks. The contrast is direct. DDGrid obtains consistency through infrastructure, whereas the model adopted here obtains determinism as a formal consequence of strong confluence. The remaining problems addressed by DDGrid fall outside the scope of this work: fault tolerance, stragglers, and scaling to thousands of tasks.

Arrighi et al. [13] showed, finally, that confluence alone does not guarantee determinism when the distributed computation does not terminate. This warning gives rise to premise P6 (Section 4).

In summary, prior works validated the properties of IC in shared memory (HVM2 [5], Kahl [9], Pinto [10]) and theoretically demonstrated the feasibility of distribution (Mackie [6]), but none combined IC with grid computing. This work positions itself in that intersection.

4 METHODOLOGY

Figure 2 summarizes the complete process of this work, from the formal theory to the experimental verification. The properties of IC motivate a set of premises; these are

formalized under *Spec-Driven Development*, implemented in Relativist, and exercised by three campaigns that verify, on each run, the equivalence between the sequential and the distributed reduction. The following subsections detail each stage in that order.

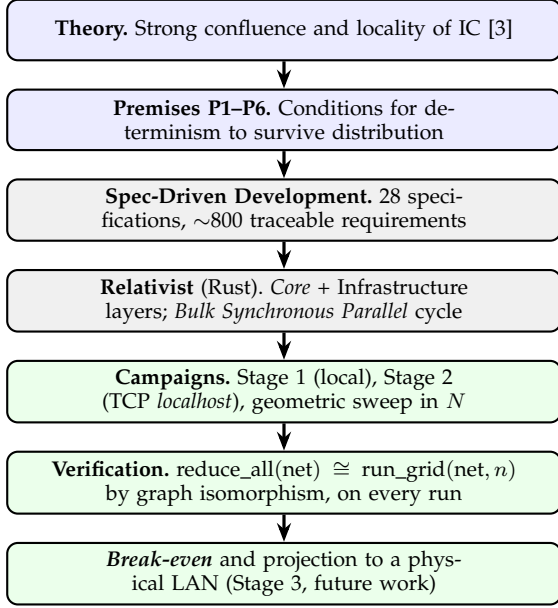


Figure 2: Overview of the process, from theory to result. The formal properties of IC motivate premises P1–P6; these are formalized under *Spec-Driven Development* and implemented in Relativist; three campaigns verify, on each run, the equivalence between the sequential and the distributed reduction by graph isomorphism; the break-even analysis projects the network-bandwidth range in which distributing becomes advantageous. Diagram produced by the author in TikZ with AI assistance.

4.1 Approach: Spec-Driven Development

The investigation adopted *Spec-Driven Development*, so that all design decisions were formalized in specifications before implementation. The systematic analysis of the literature revealed that strong confluence is necessary, but not sufficient, to guarantee determinism in distributed memory. Six premises were identified:

- **P1 — Strong confluence of the model.** Guaranteed by construction (Section 2.1).
- **P2 — Correctness of partitioning.** Each agent belongs to exactly one partition, with no loss of connections.
- **P3 — Completeness of boundary resolution.** Wires cut by the partitioning are fully restored after local reduction.
- **P4 — Consistency of identifiers.** Workers create new agents without ID collision.
- **P5 — Termination of the protocol.** The distributed reduction cycle converges in a finite number of rounds.
- **P6 — Restriction to terminating nets.** The input net reaches normal form in a finite number of interactions. Arrighi et al. [13] demonstrated that confluence

alone is insufficient to guarantee determinism in non-terminating distributed computations.

The central thesis states that, once the six premises are satisfied, the distributed reduction produces a result structurally identical to that of the sequential reduction. To make it verifiable, each premise was associated with a concrete invariant in Relativist’s specifications: strong confluence, with the theorems of the reduction core; partitioning, with the round-trip identity between partition and merge and with the equivalence of local reduction; the boundary, with the detection of redexes that cross partitions during merge; the identifiers, with the allocation of disjoint ranges per worker; termination, with the monotonic progress of the BSP cycle. The restriction to terminating nets delimits the validity scope of all the previous ones.

All premises were translated into verifiable technical specifications. Relativist v1 gathered 17 specifications (about 560 requirements); v2 (v0.20.0-pre) expanded this set to 28 specifications (about 800 requirements), adding transport abstraction, versioned wire format, *delta* protocol, *streaming* generation, arena management, and dynamic worker admission (Section 4.6). The analytical work was supported by artificial-intelligence tools, used as a critical-analysis assistant under the author’s direction. By keeping the specifications separate from the Rust implementation, it was possible to write this section even before the code was ready.

4.2 Relativist Architecture

Relativist is implemented in Rust and follows the *Bulk Synchronous Parallel* (BSP) programming model: each round of distributed reduction corresponds to a BSP *superstep* composed of local computation, communication, and a synchronization barrier. The architecture is organized into two layers with distinct responsibilities.

The **Core layer** contains the pure, synchronous logic, with no dependency on an asynchronous runtime, network, or I/O:

Table 1: Core layer modules

Module	Responsibility
net	Net representation: agents, ports, wires, allocation arena
reduction	Reduction engine: 6 interaction rules, dispatch, <code>reduce_all</code>
partition	Partitioning: allocation function, <i>FreePort</i> , ID space
merge	Recombination: union of partitions, boundary resolution
encoding	Encoders for structured workloads (polynomial evaluation via Horner’s method) and Church arithmetic primitives used internally

The **Infrastructure layer** implements asynchronous communication over *tokio* (an asynchronous I/O runtime for Rust):

Each module is formally specified.

The chosen topology is the star (Fig. 3). Resolving the boundary redexes falls solely to the coordinator, which runs `reduce_all` over the already-recombined net. This is the only moment when all agents are gathered and the `FreePort (bid)` sentinels have been resolved. The decision

Table 2: Infrastructure layer modules

Module	Responsibility
protocol	TCP protocol: messages, binary framing, serialization
config	Configuration: CLI arguments, module wiring
security	Token authentication and optional TLS
observability	Structured tracing and optional Prometheus metrics
io	Net formats, workload generators, examples
coordinator	Coordinator: FSM, BSP cycle, centralized merge
worker	Worker: reception, local reduction, result return

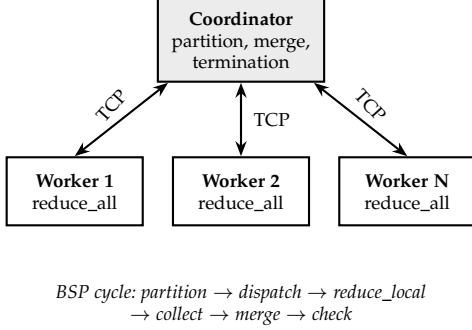


Figure 3: Star topology of Relativist. The coordinator centralizes partitioning, merge, and termination detection. Each worker runs local reduction independently, connecting directly to the coordinator over persistent TCP, with no worker-to-worker communication. Diagram produced by the author in TikZ with AI assistance.

simplifies the protocol and meets the proposed experimental validation, but limits scalability. The solution holds as long as the coordinator does not become a bottleneck.

4.3 Net and Reduction: Types and Interaction Rules

The net is represented by Lafont’s three symbols [3]: *gamma* (CON, arity 2), *delta* (DUP, arity 2), and *epsilon* (ERA, arity 0). Each agent has a unique identifier (`AgentId`, a monotonically increasing `u32`) and ports indexed by `PortId` (`u8` $\in \{0, 1, 2\}$), where 0 is the principal port and 1–2 are auxiliary. Connections are stored in a flat *port array* of size $3 \times |A|$, where the slot $\text{id} \times 3 + \text{port_id}$ holds the reference to the connected port, giving $O(1)$ access. The `PortRef` type distinguishes two cases: `AgentPort(AgentId, PortId)`, for connections between agents, and `FreePort(bid)`, used both for free ports of Lafont’s original interface [4] and as a boundary sentinel introduced by partitioning (Section 4.4).

Detection of *active pairs* is incremental: the redex queue (`VecDeque<(AgentId, AgentId)>`) is maintained by the `connect(a, b)` operation, which, on linking two principal ports, automatically inserts the pair into the queue and thus eliminates the need for an $O(N)$ scan per step.

The six interaction rules faithfully implement the topology defined by Lafont [3]:

Dispatch uses a static 3×3 table indexed by the pair’s symbols, with $O(1)$ complexity. The `reduce_all` loop consumes the redex queue until it is empty (normal form).

Table 3: The six IC interaction rules

Active Pair	Pattern	Result	Balance
CON–CON	Annihilation	Cross reconnection of auxiliaries	−2 ag.
DUP–DUP	Annihilation	Parallel reconnection of auxiliaries	−2 ag.
ERA–ERA	Void	Elimination of both	−2 ag.
CON–DUP	Commutation	4 new agents (2 CON + 2 DUP)	+2 ag.
CON–ERA	Erasure	2 new ERA on the binary agent’s auxiliaries	0 ag.
DUP–ERA	Erasure	2 new ERA on the binary agent’s auxiliaries	0 ag.

Before applying any rule, the reduction engine checks the validity of the dequeued pair through invariant I4 (*Redex Queue Validity*): both agents must exist and their principal ports must be mutually connected. Stale pairs, whose agents were consumed by earlier interactions, are silently discarded, without applying rules. An interaction counter (`u64`) accumulates the total of applied rules. Invariant T7 guarantees that this total is deterministic, independent of the reduction order.

4.4 Partitioning and Sub-net Merging

Partitioning is defined by an allocation function $\sigma : A \rightarrow \{0, 1, \dots, n - 1\}$ that assigns each agent to exactly one worker, inducing a mathematical partition of the set of agents. The implementation uses round-robin allocation ($\sigma(i) = i \bmod n$), a strategy that guarantees uniform distribution, although it does not optimize the minimization of boundary wires. Wires are classified into three categories: **internal** (both endpoints in the same partition), **interface** (involving a pre-existing `FreePort`), and **boundary** (*border wires*, with endpoints in different partitions). For each boundary wire, the `split` operation replaces the original connection with a pair of `FreePort(bid)` sentinels with a unique border identifier, registered in a *border map* (`HashMap<u32, (PortRef, PortRef)>`).

Three correctness conditions guarantee that partitioning preserves the net’s structure. Condition **C1** (complete coverage) requires every live agent to belong to exactly one partition; **C2** (wire coverage), that every wire be classified unambiguously; **C3** (`FreePort` bijectivity), that each `bid` appear as a `FreePort` in exactly two distinct partitions. Together, these conditions formalize invariant D1 (*Split/Merge Identity*): $\text{merge}(\text{split}(\text{net})) \cong \text{net}$, the operational basis of premise P2. Each worker also receives a contiguous and exclusive range of `AgentId`, guaranteeing premise P4 (consistency of identifiers) without runtime coordination.

The `merge` operation reconstructs the net from the reduced partitions: it unites the agents, copies the internal connections, and restores the boundary connections by consulting the border map. The restoration invokes `connect(a, b)`, which automatically detects new active pairs when both ports are principal, including emergent boundary redexes (which did not exist in the original net but arose after local reduction).

4.5 Coordination Protocol and Grid Computing

The distributed reduction cycle follows the BSP model in six phases: (1) **partitioning**, in which the coordinator applies `split` to the global net; (2) **dispatch**, in which each partition is sent to the corresponding worker over TCP; (3) **local reduction**, in which each worker runs `reduce_all` over its partition; (4) **collection**, in which the workers return the reduced partitions; (5) **merge and boundary resolution**, in which the coordinator recombines the partitions and resolves boundary redexes via `reduce_all`; (6) **termination check**: if the redex queue is empty, the net has reached normal form; otherwise, a new round begins.

Phase 5 handles a subtle case, that of emergent *border redexes*. Consider the CON-DUP rule (commutation). When applied locally, it consumes two agents and creates four new ones. If one of the consumed agents had a port connected to `FreePort (bid)`, the new agent inherits that connection. After the merge, when the sentinel is resolved to the corresponding agent in the other partition, a new active pair may form that did not exist in the original net. Detection occurs in the `connect(a, b)` operation itself. On reconnecting the two ports of a `bid`, it checks whether both are principal ports and, if so, enqueues the pair as a redex. The mechanism is iterative: the coordinator’s `reduce_all` consumes all emergent border redexes in the same phase, without the need for a new BSP round.

The communication protocol uses persistent TCP connections between coordinator and workers, avoiding the overhead of a handshake per round. The framing uses an 8-byte header (4 bytes for the payload length in little-endian and 4 bytes for a CRC32 checksum, a 32-bit *Cyclic Redundancy Check*), followed by the payload serialized via *serde* and *bincode* (binary serialization libraries of the Rust ecosystem). Four message types make up the catalog: `AssignPartition` and `Shutdown` (coordinator to worker), `PartitionResult` and `Error` (worker to coordinator).

Termination of the protocol (premise P5) follows from the finiteness of terminating nets. Each round consumes at least one interaction of the total deterministic budget (invariant T7), so the loop converges. The BSP barrier guarantees that the coordinator runs the merge only after receiving all partitions, preserving conditions C1–C3 between rounds.

4.6 v2 Architecture Evolutions

The initial version of Relativist (v1, frozen at v0.10.0-bench) established the functional base described above. From it, six architectural evolutions were specified and implemented to empirically support premises P2–P5 on larger nets and under conditions closer to a real grid. All followed the same *Spec-Driven* cycle: formal specification → adversarial review → TDD implementation → unified review → adversarial QA.

Transport abstraction. The direct coupling between the coordinator and the asynchronous runtime was replaced by an interface that isolates the logical protocol from the physical medium, so that the transport can be swapped without reopening the proof obligations of the logical layer.

Wire format v2. *bincode* serialization remains the default, complemented by an optional zero-copy path (*rkyv*); the protocol version is negotiated in the handshake.

Delta protocol. Instead of retransmitting each reduced partition in full, the worker sends only the difference relative to the state known to the coordinator (consumed agents, created agents, new wires). The academic relevance is direct, since the feasibility of grid computing depends on the communication cost not canceling the parallelism gain. Without the delta protocol, each BSP round would send $O(|A|)$ bytes per worker; with delta, traffic scales with the size of the active boundary rather than with the total size of the net. This evolution is the empirical core verified in Section 5.

Streaming generation. The coordinator no longer materializes the complete net before partitioning. The input is processed in chunks, and the workers request work as they consume it, which removes the coordinator’s RAM as an architectural bottleneck.

Arena management. The allocation arena gained stack-based identifier recycling and an alternative sparse representation, keeping memory use bounded under long-running streaming. The choice between representations uses an efficiency metric defined as the ratio between useful memory and the allocated identifier range.

Elastic grid. The coordinator was also promoted to a worker, and new workers may join in later rounds. This elasticity is an operational corollary of strong confluence. Since the result is independent of who reduces each active pair, adding capacity mid-execution does not invalidate correctness.

Together, these six evolutions do not change the central thesis, since premises P1–P6 remain, but they create the empirical infrastructure that allows, in Section 5, separating the cost of distributed reduction into two parts: the cost of the model (partitioning and merge) and the cost of transport (network, framing, and serialization).

4.7 Experimental Design

The benchmark suite is organized into three overhead profiles, derived from the distinction between batch and streaming workloads of Mackie and Sato [11] (Section 3):

Table 4: Overhead profiles of the benchmark suite. The *Border* column indicates emergent border redexes per round.

Profile	Description	Benchmarks	Rnd.	Border
A	Emb. parallel	EP-ERA, EP-CON, EP-DUP	1	0
B	Expansion + collapse	CON-DUP Exp., MixedNet, ChurchAdd, ChurchMul	Var.	Emerg.
C	Sequential dep.	DualTree	d (depth)	~50%/lvl.

Profile A gathers the most favorable scenarios. All redexes are independent and resolve in a single round, which makes the scenario ideal for distributing. Profile B includes the CON-DUP rule, the only one that makes the net grow,

and therefore generates new border redexes at each round; it is the profile closest to a real application, in which the net first grows and then collapses. It includes the `ChurchAdd` and `ChurchMul` benchmarks, which exercise the Church-numeral arithmetic used as a primitive by the structured-load encoders. The empirical evidence that Lafont’s universality [3] holds in practice, within the scope of this work, is the demonstration of Horner’s method (Section 5.7). There, polynomial evaluation is written entirely with the six interaction rules, without auxiliary mechanisms. Profile C is the worst case, with strong sequential dependence among the levels of the tree. `TreeSum` enters as a data-bound case of pragmatic encoding (precomputed sum, ERA–ERA pairs as units of work), which validates the distribution machinery without requiring the full formal expressiveness of the model.

Each benchmark is run with 1, 2, 4, and 8 workers, with logarithmic variation of the input net size. The fundamental property verified at **each data point** is:

$$\text{reduce_all}(\text{net}) \cong \text{run_grid}(\text{net}, n) \quad (1)$$

where $\cong$ denotes graph isomorphism, that is, structural equality up to the renaming of identifiers. The verification canonicalizes the identifiers deterministically and compares the structures. The success criterion is the absence of correctness failures across the whole suite. Beyond correctness, the following are recorded: speedup (sequential time divided by the distributed time), MIPS (millions of interactions per second), overhead ratio ($T_{\text{overhead}}/T_{\text{total}}$), border ratio (boundary redexes over the total), efficiency (speedup over w), and the break-even point, the smallest size from which distributing is advantageous. The clock is monotonic (`std::time::Instant`); each data point is the median of 10 repetitions after 2 warmup rounds, accompanied by 95% confidence intervals for the times.

The design measures the equivalence between the sequential and the distributed reduction and decomposes the cost of distribution, but does not include a direct performance comparison with other systems. The omission is deliberate. The model evaluated here is a still-early paradigm of distributed reduction, whereas the grid-computing or parallel-evaluation platforms it could be compared against have accumulated one to two decades of engineering optimization. An absolute-time comparison between a reference prototype and those mature platforms would measure mostly the difference in implementation maturity, not the viability of the model. For that reason, the evaluation concentrates on two questions that are independent of that asymmetry: does the distributed reduction preserve the result of the sequential one, and how does the cost of distribution decompose and scale with the size of the net. The qualitative positioning against the literature is resumed in Section 5.

5 RESULTS AND DISCUSSION

5.1 Experimental Environment

The experiments were run on a Lenovo ThinkPad T14 Gen 4 (Intel Core i7-1365U, 2 P-cores at 5.2 GHz + 8 E-cores at

3.9 GHz, 32 GB DDR5-5200, Windows 11 Pro). Three hardware caveats are documented for transparency: (1) the U-series processor sustains PL1 = 15 W indefinitely, with turbo PL2 = 55 W exhausted in seconds, so that long benchmarks operate in the throttled regime; (2) the Windows hybrid P/E scheduler distributes threads between performance and efficiency cores, introducing variance across repetitions (4 data points with CV between 0.155 and 0.193); (3) in Stage 2, Docker Desktop (WSL2) allocates 12 vCPUs over 10 physical cores. These limitations are acceptable because Stage 3 on a LAN will compare the *same hardware* against physical machines on a network, isolating the network cost by subtraction.

The baseline campaign used the single binary `v0.10.0-bench` (`rustc 1.94.1`, release profile), under the measurement protocol of Section 4.7. Variance was analyzed by CV triage (threshold $\text{CV} \leq 0.15$). Of the 420 summary points, 63 were flagged (58 by sub-millisecond timer noise, 4 by P/E scheduling, 1 by Docker noise) and all classified as *keep*, with no exclusion or re-execution.

The system is written in Rust (2021 edition, release profile), compiled with the `rustc 1.94.1` toolchain; the exact versions of all dependencies, including the transitive ones, are pinned in the versioned `Cargo.lock`, and Table 13 (Section 5.10) consolidates the provenance for reproduction. Concurrency and transport use `tokio 1.x`, a multi-thread asynchronous runtime that serves the TCP protocol, and `rayon 1.x`, which provides the local parallelism used in the fair comparison between sequential and distributed reduction. State serialization combines `serde 1.x` with `bincode 2.x` for the wire format; `crc32fast` verifies the integrity of each frame, and `lz4-flex 0.11` compresses the frame before transmission. Measurement and testing rely on `criterion 0.5`, which reports times with a 95% confidence interval, and on `proptest 1.x`, which generates the property-based tests of the correctness suite.

Beyond the baseline campaign (3800 + 400 repetitions), a **stress campaign** extended the net sizes to production scale: `ep_annihilation_con` up to 50 million agents and `dual_tree` up to depth 25 (~67 million agents). The same binary, hardware, and methodology were used, with 5 repetitions per configuration (per-repetition cost of 10–80 s at stress scale). The stress campaign lasted ~67 min (Stage 1) and ~106 min (Stage 2), producing 290 additional runs. In total, **4490 runs** were carried out across the two campaigns.

5.2 Stage 1 Results — Local Reduction (In-Process)

Stage 1 exercised 12 benchmarks in *lenient* mode and 2 in *strict* mode, varying the workers over {sequential, 1, 2, 4, 8}, for a total of 3800 repetitions in 11 min 39 s. **No correctness failure occurred**: at each data point, the fundamental property (Equation 1) was confirmed by graph isomorphism.

Table 5 summarizes the median times for the representative benchmarks of each profile.

The main finding of Stage 1 is that, in local mode, there is no break-even. Of the 272 speedups measured with the local grid, 44.9% were below 0.1, that is, the grid was more than $10\times$ slower than the sequential, and 78.4% did not exceed 0.5. Only 4 configurations exceeded speedup = 1.0, all with a single worker: the gain came from the cache of

Table 5: Stage 1: median wall-clock times (seconds) per worker configuration. All points with verified correctness.

Benchmark	Seq. (s)	w=1	w=2	w=4	w=8
ep_con 100k	0.013	0.014	0.034	0.036	0.044
dual_tree 14	0.001	0.001	0.005	0.005	0.006
condup_exp 5k	0.001	0.001	0.001	0.002	0.002
cascade_cross 1k	0.000	—	—	—	0.000

a compact subnet, not from real parallelism. **With two or more workers, no configuration beat the sequential.** The reason is twofold. When communication is instantaneous, the $O(|A|)$ cost of partitioning and recombining dominates the total time; and, with only 2 physical P-cores on the i7-1365U, any configuration of three or more workers contends for the same cores.

Production-scale validation. To check whether the baseline findings hold on larger nets, the stress campaign took `ep_annihilation_con` to 50M agents (corresponding to 100M input nodes) and `dual_tree` to depth 25 (~ 67 M agents), totaling 150 repetitions in about 67 min. Again, **no correctness failure** occurred at any point, including on the 50M-agent net.

Table 6 shows the median times and the slowdown ratio with 2 workers.

Table 6: Stress campaign (Stage 1): median wall-clock times (seconds) at production scale. `wall_clock_median` values from the frozen CSVs. 5 reps/config, 2 warmup.

Benchmark	Seq.	w=1	w=2	w=4	w=8	w=2/Seq
ep_con 10M	3.47	3.54	9.64	10.68	13.27	2.78×
ep_con 20M	7.37	7.62	20.67	23.17	28.67	2.80×
ep_con 50M	20.45	20.76	54.49	60.47	76.01	2.66×
dual_tree 23	1.55	1.57	7.04	7.12	7.30	4.54×
dual_tree 24	3.52	3.50	14.75	14.93	15.42	4.19×
dual_tree 25	7.44	7.49	31.00	31.16	32.14	4.16×

The most significant result is the **scale invariance** of the relative overhead. In `ep_annihilation_con`, the $w=2/\text{Seq}$ ratio varies little: $2.78\times$ at 10M, $2.80\times$ at 20M, and $2.66\times$ at 50M, less than 5% of variation. In `dual_tree`, the pattern repeats, with $4.54\times$ at $d=23$ and $4.16\times$ at $d=25$. The conclusion is that the cost of distributing is **proportional to the net size**, not a fixed startup cost that dilutes as the net grows. The cause is that each BSP round reconstructs and reduces the entire net at the coordinator, an operation that scales linearly with $|A|$.

This cost proportional to the net size is consistent with the lesson of Mackie [6], for whom partitioning dominates the performance of distributed interaction-net reduction. The contrast with HVM2 [5] delimits what is claimed here. That evaluator obtained near-linear speedup, but in shared memory, where there is no transport or recombination. This work does not claim a parallelism gain in localhost, where the transport and the merge at the coordinator outweigh the useful reduction work. The advantage of the distributed model is restricted to the bandwidth-limited network regime, examined in Section 5.5.

Validation of strict BSP mode. The *strict* mode confirms the theoretical predictions for the number of rounds with an exact match:

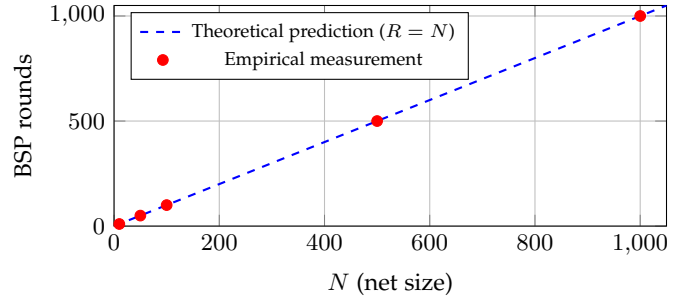


Figure 4: Validation of strict BSP mode: the empirical rounds of `cascade_cross(N)` coincide exactly with the theoretical prediction $R = N$ for $w \geq 2$ workers. Diagram produced by the author in TikZ/pgfplots with AI assistance.

`cascade_cross(N)` terminates in exactly N rounds and `dual_tree(d)` in d rounds (with workers ≥ 2), confirming that the BSP protocol preserves the semantics of distributed reduction and that the detection of emergent border redexes works correctly between rounds.

5.3 Stage 2 Results — TCP Localhost under v2

Relativist v2 (the v0.20.0-pre pre-release of Section 4.6) was re-evaluated under TCP *localhost* protocol in Docker containers, exercising at once the streaming generation, the delta transfer protocol, and the memory arena management. The campaign gathers 32 distributed configurations, each with 10 repetitions, plus 8 native sequential baselines, each with 10 repetitions, for a total of 400 data points (320 distributed and 80 sequential). The 32 configurations confirmed structural correctness in 100% of the repetitions. Together with those of v1 (4 490 runs, of which 4 465 completed the verification; the remaining 25 aborted on a TCP frame limit, as discussed under Limitations), these numbers total about 4 800 distributed reductions verified against the sequential reduction, without a single correctness divergence. Table 7 summarizes the median times.

Table 7: Stage 2 (v2): median wall-clock times (seconds) over TCP *localhost* in Docker, campaign `v2_post_d012_baseline_2026-05-05`. All points with `all_correct=true`.

Benchmark	Seq. (s)	w=1	w=2	w=4	w=8
ep_con 500k	0.368	0.460	0.575	0.680	1.135
ep_con 1M	0.576	0.935	1.172	1.339	2.043
ep_con 5M	3.185	5.815	7.403	7.959	11.805
dual_tree 18	0.288	0.296	0.448	0.518	0.749
dual_tree 20	0.483	0.830	1.438	1.605	1.942
dual_tree 22	1.463	3.674	6.335	7.227	8.153

v2 vs. v1 comparison. The same configurations were re-evaluated on the frozen v1 binary (v0.10.0-bench), allowing a direct v2/v1 ratio per benchmark family:

The interpretation of this result depends on the communication regime. v2 costs $\sim 1.6\text{--}1.8\times$ more on localhost to deliver the same normal form; Section 5.5 shows why this overhead turns into an advantage on a real network.

Effectiveness of the delta protocol. Despite the additional wall-clock cost, v2 transmits fewer bytes over the net-

Table 8: v2/v1 ratio of median wall clock over TCP *localhost*.
[†] `condup_expansion` operates at very small scale (1 k–5 k agents) and absolute times below 60 ms; the ratio reflects fixed handshake and framing cost amortized over a tiny load and does not generalize to production regimes.

Family	N slots	v2/v1 median	Range
ep_annihilation_con	12	1.63×	1.43 – 2.06
dual_tree	12	1.77×	1.26 – 3.04
condup_expansion [†]	8	12.8×	7.04 – 25.4

work than v1 sends in a single direction, because the delta protocol carries differences, not complete states. For the `ep_annihilation_con` 500 k configuration with $w=1$, on the first round:

Table 9: Bytes transferred per round on the coordinator–worker path. Although v2 introduces bidirectional traffic (return delta), the total is 30% lower than v1’s unidirectional traffic. Source: `rounds.csv`, `ep_annihilation_con` 500 k $w=1$, round 0.

Version	Coord. → W	W → Coord.	Total
v1	41.00 MB	< 1 KB	41.00 MB
v2 (<i>delta</i>)	23.80 MB	4.89 MB	28.69 MB
Change	–42%	—	–30%

The asymmetry of v1 traffic (41 MB out, ~ 153 B back) stems from the coordinator sending the complete partition and receiving only a result identifier; v2, by construction, also sends a return delta. The fair comparison is over the total transferred, 30% smaller in v2.

Historical v1 campaigns preserved. Table 5 (Stage 1 in-process) and Table 6 (v1 stress) remain valid as v1 references, since the evolutions of Section 4.6 introduced new code paths that did not exist in v1, and therefore the scale ratios observed in v1 were not retroactively altered.

5.4 Per-Round Cost Decomposition

The instrumentation introduced in this phase of v2 exposed, for the first time, the per-component times in each BSP round, namely partitioning, local computation, merge, and network, all collected per worker and reported to the coordinator via delta. Table 10 presents the per-component average over 320 distributed TCP *localhost* rounds of the canonical campaign.

Table 10: Average per-round decomposition over TCP *localhost* (v2, 320 rounds). The network component dominates even on loopback, because v2 negotiates bidirectional state every round. The residual concentrates framing, FSM orchestration, and arena operations.

Component	Mean time (s)	Fraction
Partitioning (<code>split</code>)	0.0009	0.04%
Local computation (<code>reduce_all</code>)	0.192	8.0%
Merge at coordinator	0.145	6.0%
Network (framing + TCP)	1.139	47.3%
Residual (orchestration + allocation)	0.933	38.7%
Measured total	2.409	100%

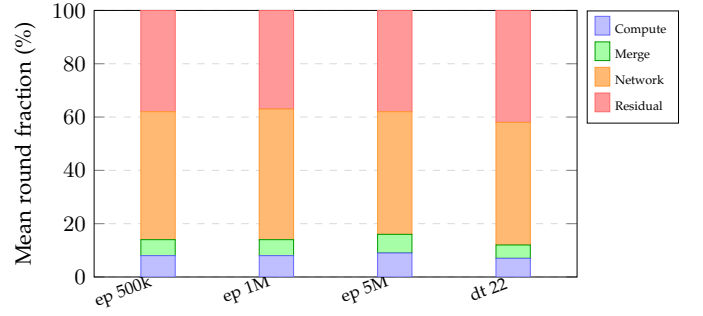


Figure 5: Average percentage decomposition of the time per round over TCP *localhost* (v2). The partitioning component ($< 0.1\%$) is omitted for visual feasibility. Network dominates systematically ($\sim 46\text{--}49\%$), followed by the orchestration residual; compute and merge add up to less than 15%. Diagram produced by the author in TikZ/pgfplots from `rounds.csv`, with AI assistance.

The indirect estimate from v1 (Stage 2 / Stage 1 ratio) gives way to a direct measurement. Over TCP *localhost*, about 47% of each round goes to the network, with no physical network in the path. This time corresponds to the TCP kernel, the bincode framing, and the serialization of the delta; on a physical LAN, part of this work would occur in parallel with the actual transmission. The residual ($\sim 38\%$) gathers the state machine of the streaming generation, the arena operations, and the header framing. Compute and merge, combined, stay below 15% of the time. Reducing the net, the very work of Lafont’s model, is not the bottleneck.

5.5 Break-Even Analysis and LAN Projection

v1 of this work empirically derived, from Stage 1 and Stage 2, a ratio $c_o/c_r \approx 2.2$ between the overhead cost of distribution and the local reduction cost. v2 refines this analysis with per-component data.

Relative to v1, v2 pays approximately **100 ms of additional CPU per round** (transport and framing layer) and **saves ~ 12 MB per round on the wire** (Table 9). The bandwidth break-even is the point from which the byte savings outweigh the CPU cost. It follows in two steps. The naive floor divides the byte savings by the gross CPU cost; discounting the header-framing portion ($\sim 30\%$ of the payload), which does not contend with the useful transmission, the CPU time effectively attributable to the exchange drops to ~ 77 ms and raises the equilibrium point:

$$B_{\text{eq}}^{\text{gross}} = \frac{\Delta \text{ bytes}}{\Delta \text{ CPU time}} = \frac{12 \text{ MB}}{0.100 \text{ s}} = 120 \text{ MB/s},$$

$$B_{\text{eq}} = \frac{12 \text{ MB}}{0.077 \text{ s}} \approx 156 \text{ MB/s} (\approx 1.2 \text{ Gb/s}). \quad (2)$$

v2 tends to beat v1 on networks slower than 156 MB/s, a range that covers the typical bandwidth of a home LAN and of a WAN. On localhost, where bandwidth is, in practice, unlimited, the trade-off is not justified and v2 loses, as Tables 7 and 8 show. Hence localhost is not the appropriate regime to evaluate this optimization, since the gain of the delta protocol depends on a transmission time that, there, does not exist.

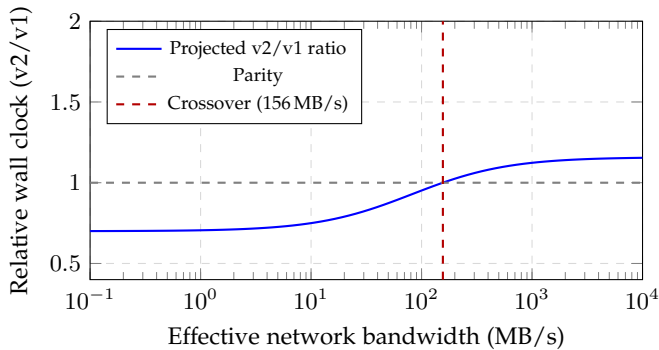


Figure 6: Projection of the $v2/v1$ ratio as a function of effective network bandwidth, derived from the decomposition of Table 10 and the byte volume of Table 9. To the right of the crossover (high bandwidth, localhost), $v2$ loses; to the left (typical LAN, ~ 100 MB/s; WAN, ~ 10 MB/s), $v2$ wins. The empirical measurement on a physical LAN (Stage 3) is the object of Section 6.3. Diagram produced by the author in TikZ/pgfplots, with AI assistance.

This crossover is the empirical signature that the work’s theoretical argument required. It is the bandwidth point at which the distributed layer ceases to be mere overhead, now decomposed into measurable components. It is derived, not measured. Direct confirmation depends on execution on a physical LAN, addressed in Section 6.3.

5.6 Extended Verification under a Geometric Sweep of N

The Stage 1 and Stage 2 campaigns reported so far sample, for each benchmark, six size points up to about 50 M agents. To observe the central property along a **curve**, and not at isolated points, an auxiliary campaign swept the N dimension more finely and more broadly, over a single representative load, the CON-CON annihilation (*ep_annihilation*). The sizes were spaced geometrically by a factor of $\sqrt{10}$, from 10^4 to 10^7 , in seven points covering four orders of magnitude. The same binary ran in two environments, in-process reduction and localhost over TCP in containers, each with four degrees of parallelism ($W \in \{1, 2, 4, 8\}$) and sequential reduction as the baseline.

The central datum is that **the 245 runs of the local arm and the 240 of the TCP arm all converged to the same normal form as the sequential reduction**, with no structural divergence across four orders of magnitude of N and four parallelism configurations. Figure 7 shows the wall-clock time as a function of N in the local arm (log-log axes) and Table 11 summarizes the TCP arm. In both environments, the sequential and distributed curves rise with N while keeping a constant separation. The relative overhead of the distributed layer does not increase with N ; it is multiplicative, in agreement with the analysis of Section 5.5.

The campaign has three limitations. The first is the **single load**. Three loads were originally planned for this sweep: CON-CON annihilation, CON-DUP commutation, and DUP-DUP expansion. Only the first was executed in

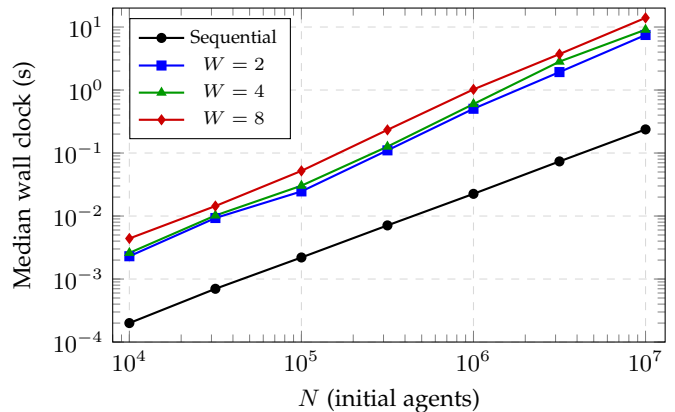


Figure 7: Median wall-clock time as a function of N in the geometric sweep (*ep_annihilation*, in-process arm). Axes in logarithmic scale. The distributed curves ($W \in \{2, 4, 8\}$) follow the sequential curve while keeping a stable multiplicative separation across four orders of magnitude; structural correctness was verified at 100% of the 245 points. Diagram produced by the author in TikZ/pgfplots, with AI assistance.

Table 11: Geometric sweep in the TCP *localhost* arm (Docker): median wall-clock time (seconds) per parallelism configuration. Intermediate rows of the $\times\sqrt{10}$ progression omitted for brevity; the seven full points are in the frozen campaign. At point $N = 10^7$, the Docker arm aborted after three consecutive attempts (likely container memory exhaustion), an operational limitation of the environment and not of the protocol; correctness was verified at 100% of the 240 points that completed.

N	Seq.	$W=1$	$W=2$	$W=4$	$W=8$	
10^4	0.0002	0.0003	0.0033	0.0035	0.0051	
10^5	0.0023	0.0028	0.0563	0.0549	0.0548	
10^6	0.0216	0.0257	0.9307	0.9899	1.0958	
3.16×10^6	0.0768	0.0914	3.7627	4.2484	4.7431	
10^7	—	abort (container instability)				

this delivery; extending to the other two is listed in Section 6.3. The second is the **ceiling** at $N = 10^7$. The cut results from an operational protection against excessive memory allocation; the original extension targeted 10^9 , which would require about 68GB of RAM per run. The third is the **abort of the TCP arm at $N = 10^7$** , caused by container instability under maximum load, not by the protocol layer. An additional technical detail arose in the $N = 10^7$, $W = 8$ configuration of the in-process arm. The round-robin partitioning by `AgentId` ranges left some workers with empty sub-nets. This triggered the “empty-Net” fallback path, with no harm to correctness, since all repetitions converged to the same normal form, but it showed that the default partition is not ideal for ultra-regular loads at this scale (Section 5.5 and Limitations, below). Together, these 485 runs reinforce the evidence of the main campaigns and ground the discussion of the premises that follows.

5.7 Structured-Load Demonstration: Polynomial Evaluation

The previous campaigns exercise **synthetic** loads, namely CON-CON annihilation, CON-DUP commutation, and dual trees, all designed to isolate specific interaction rules. They answer the question of structural correctness, but leave one open: does the central property also hold under a load that encodes a **real algorithm**, whose result can be checked against an external arithmetic oracle? To answer, a demonstration based on **Horner’s method** for polynomial evaluation was implemented. The method uses the nested form $((c_n \cdot x + c_{n-1}) \cdot x + \dots) \cdot x + c_0$, which solves a degree- n polynomial with n multiplications and n additions. The net encodes that sequence of operations directly over Church numerals; decoded, its normal form must coincide with the expected arithmetic value.

The battery gathers ten distinct polynomials (Table 12), with degree from zero to two, from the trivial case (the constant 42) to cases with an evaluated value in the millions (the polynomial $10\,000 + 1025x$ at $x = 10\,000$ yields 10 260 000). Each polynomial was run at four degrees of parallelism ($W \in \{1, 2, 4, 8\}$) and two environments, in-process, in the same process, and localhost over TCP in containers, totaling **80 runs** (10 polynomials $\times$ 4 workers $\times$ 2 environments). Each run provides two independent correctness tests: the **numeric value** read from the normal form, checked against the arithmetic oracle, and the **structural normal form** of the distributed arm, compared by graph isomorphism to that of the in-process sequential arm (this second comparison does not apply to the in-process arm itself, which serves as the reference).

Table 12: Battery of Horner demonstrations: ten polynomials covering degrees 0 to 2 and varied magnitudes of coefficients and evaluation point. Each row was run in four parallelism configurations ($W \in \{1, 2, 4, 8\}$) and two environments (in-process and TCP in containers), totaling 80 runs. The **Inter.** column reports the number of elementary reduction interactions applied to reach the normal form, a measure of the net’s computational work, independent of wall-clock time. The value decoded from the normal form matched the expected one in 100% of the runs; in the 40 runs of the distributed arm, the normal form matched structurally that of the sequential arm in 100% of the cases.

#	Deg.	Polynomial	x	Expected	Inter.
D1	0	42	99	42	0
D2	1	$1 + x$	5	6	11
D3	1	$100 + x$	50	150	11
D4	1	$42 + x$	10 000	10 042	11
D5	1	$1 + 1025x$	10 000	10 250 001	2059
D6	1	$10\,000 + 1025x$	10 000	10 260 000	2059
D7	2	$1 + x + x^2$	2	7	26
D8	2	$1 + x^2$	3	10	28
D9	1	$3 + 5x$	4	23	19
D10	2	$10\,000 + 500x + x^2$	100	70 000	1220

Both criteria were met. In the **80 runs**, the value decoded from the normal form was exactly the expected one and, in the 40 of the distributed arm, the structural normal form was isomorphic to that of the in-process sequential arm.

Thus the central property of this work, the equivalence between sequential and distributed reduction, also holds for a meaningful load, confirmed by two independent paths: an external numeric oracle *and* the internal graph isomorphism. The number of interactions grows with the magnitude and degree of the polynomial, from zero (the constant, already in normal form) to 2059 (the largest-magnitude case of the linear arm). This indicates that the net *computes* the polynomial, and does not merely collapse to an embedded constant.

A scope clarification is in order. In all 80 runs, the wall-clock time stayed below what the clock can measure (less than 10 ms), and the maximum of interactions (2059) is several orders of magnitude smaller than that of the main campaigns. For that reason, **this is a qualitative demonstration, not a performance measurement**. It shows that the distributed encoder/decoder works over a structured load and that the structural equivalence holds under distribution, but says nothing about efficiency in this regime. The demonstration operates within a defined envelope. Each coefficient and the evaluation point x are represented by Church numerals, a unary encoding whose cost in agents grows with the value. For that reason, the encoder limits each input scalar to 10 000; the decoder, in turn, reads back the result for polynomials of degree up to two (at degree one, with a slope coefficient up to 1025; at degree two, with a unit leading coefficient). Within this envelope, the decoded evaluated value has no ceiling of its own: the case $10\,000 + 1025x$ at $x = 10\,000$ is correctly read as 10 260 000. Widening the decoder’s envelope to larger degrees and coefficients is one of the fronts of future work (Section 6.3).

5.8 Validation of Premises P1–P6

The six premises that support the work’s hypothesis were validated by the experimental data:

- **P1 (Strong confluence):** guaranteed by construction [4, Proposition 1] (Section 2.1).
- **P2 (Correctness of partitioning):** the identity $\text{merge}(\text{split}(\text{net})) \cong \text{net}$ confirmed at 100% of the 4465 v1 points and the 320 distributed v2 points (mechanism in Section 4.4).
- **P3 (Boundary completeness):** no boundary redex lost in the multi-round runs (detection described in Section 4.4).
- **P4 (Consistency of identifiers):** guaranteed by the allocation of disjoint ranges (Section 4.4).
- **P5 (Termination of the protocol):** measured rounds coincide with the theoretical prediction (Fig. 4).
- **P6 (Terminating nets):** all benchmarks terminate by construction.

The fundamental property (Equation 1) was satisfied at **100% of the 4465 runs with verified correctness** of v1 and at **100% of the 320 distributed runs of v2**, that is, about 4800 reductions verified against the sequential reduction, with no structural divergence.

5.9 Limitations

- 1) **Absence of validation on a physical network.** Over TCP *localhost*, the effective bandwidth is above the

156 MB/s crossover derived in Section 5.5, so that neither v1 beats the sequential in this regime, nor does v2 beat v1. The region where distribution is advantageous, that is, a real LAN at ~ 100 MB/s, is the object of Stage 3 (Section 6.3), whose operational protocol accompanies the v0.20.0-pre pre-release of Relativist.

- 2) **Round-robin partitioning.** The current strategy does not optimize the minimization of boundary wires (Section 6.3).
- 3) **Single coordinator, star topology.** Scalability is limited by the bandwidth of the centralized merge. v2 already introduces, in detection mode, dynamic worker admission and the use of the coordinator itself as a worker. The complete path of reincorporating a worker after departure is addressed in Section 6.3.
- 4) **Benchmark hardware.** The absolute times reflect an ultrabook with thermal throttling. The ratios (speedup, efficiency) are more reliable than the absolute values.
- 5) **TCP frame limit resolved in v2.** v1 limited each TCP frame to 1 GiB, aborting 25 runs of the stress campaign. v2 resolved this limitation through streaming generation (Section 4.6).

5.10 Reproducibility

All experimental artifacts of this work are public and verifiable. The binary used in the Stage 1 and Stage 2 campaigns of v1 is the v0.10.0-bench tag of the public Relativist repository; the v2 evolution is the v0.20.0-pre pre-release of the same repository. Three campaigns are packaged and frozen in the repository: Stage 1 in-process, Stage 2 over TCP *localhost* under Docker containers, and the geometric sweep in N . Each carries a manifest that records provenance (binary version, environment, exact commands) and the cryptographic checksums of the raw data. The recipe to reproduce the battery on a physical LAN accompanies the same pre-release as an appendix (Section 6.3). Table 13 consolidates the minimal items to reproduce the campaigns.

Table 13: Provenance for reproduction. The toolchain was recorded in the frozen campaigns’ manifests; the dependencies, including the transitive ones, are pinned in the versioned `Cargo.lock`.

Item	Value
Toolchain	<code>rustc 1.94.1 (2026-03-25)</code> / <code>cargo 1.94.1</code> ; 2021 edition, release profile
Binaries	<code>v0.10.0-bench (v1)</code> , <code>v0.20.0-pre (v2)</code>
Dependencies	exact versions (with transitive ones) pinned in the versioned <code>Cargo.lock</code>
System	Windows 11 Pro; Stage 2 in Docker Desktop containers over WSL2
Raw data	three frozen campaigns, each with a provenance manifest and SHA-256 checksums

6 CONCLUSION AND FUTURE WORK

6.1 Synthesis and Contributions

The hypothesis of this work was confirmed at every measured point. It states that the strong confluence of IC,

combined with a protocol that preserves the net’s structure (premises P1–P6, Section 4), leads to a distributed normal form identical to that of the sequential one on terminating nets. Across the 5350 reductions verified between the main campaigns, the geometric sweep in N , and the Horner demonstration (the only one checked by both an external numeric oracle *and* graph isomorphism), the result was 100% of identical normal forms (Section 5). The consistency covers four orders of magnitude in net size (10^4 to 10^7 agents), up to eight workers in parallel, and holds for synthetic and structured loads.

From this evidence, the work extracts four contributions. The **theoretical** one is to identify a coherent set of operational premises that distributed reduction must satisfy for Lafont’s strong confluence [3] to turn into determinism of result on a grid. These are P1 to P6 of Section 4, each associated with a verifiable invariant in the implementation’s specifications. The **engineering** one is Relativist itself, built under *Spec-Driven Development* and published as an operational reference of the model, open-source and reproducible. The **empirical** one is the frozen battery of experiments. To date, this is the first empirical validation of this equivalence between sequential and distributed reduction of IC on a grid architecture. The **methodological** one, recorded in Section 6.2, shows how the formal invariants and the empirical tests complement each other during periods of aggressive refactoring.

It remains to delimit what this evidence proves and what it does not. What is asserted is the **structural equivalence** between the sequential and the distributed reduction, the central property of the hypothesis, confirmed by graph isomorphism in all completed runs. This confirmation does not depend on the physical topology of the network. It suffices that the protocol preserve the net’s structure from one round to the next; for that reason, even the localhost result supports the thesis. What is missing is the **performance characterization on a real network**. Since all experiments ran on localhost (a single process and TCP containers on a single host), the break-even analysis (Section 5.5) only projects, without measuring on a LAN, from what bandwidth distribution becomes advantageous. This is the starting point of future work.

6.2 Formal Invariants as an Orthogonal Coverage Layer

A methodological finding emerged, unplanned, in the transition between Relativist versions v1 and v2. On re-evaluating the TCP *localhost* campaign under v2, a wall-clock regression was observed on a representative load (CON–CON annihilation, five million agents, two workers), from about 14.2s in v1 to near 22s, an 83% increase.² A seven-point bisection of the history pointed to the cause. In a refactoring of the partition layer, an arena-size metric started measuring the planned identifier range, and not the space occupied by live agents, so that valid partitions were routed to an inefficient fallback path. Fixing the routing

2. The absolute values cited in this subsection come from a reproducibility harness that runs the binary directly in a native environment. They differ from the values reported in Table 7 (Section 5), obtained in a Docker container with warmup normalization and discarding of the first repetition. The $1.11\times$ ratio between v1 and post-fix v2, cited at the end of the subsection, is stable across the two harnesses.

exposed **two latent bugs** in the optimized path, introduced by the same refactoring and absent in v1: one compromised the incremental transfer of deltas; the other allowed the coordinator to reuse identifiers already allocated to a remote worker. The second was caught at runtime by the automatic predicate that monitors net consistency, when it collided on an adjacent load; the first, by an adversarial reading of the worker–coordinator payload contract. The specification was amended (metric corrected, two guards at the agent-creation and merge points); each bug was fixed with two lines of code, and the post-fix campaign returned the ratio to $1.11 \times$ relative to v1, within the measurement noise floor.

The lesson is not that v2 is, *a posteriori*, more correct than v1. The two bugs were born in v2’s own refactoring. Nor is it to claim that formal invariants detect what empirical tests do not. Property-based tests, with broad enough load coverage, would probably capture most of these cases. The defensible point is more modest and **complementary**. The invariant predicates do not depend on the load and fire the instant the intermediate state violates the property, without requiring the author to foresee the adversarial space; the property tests cover the *load* $\times$ *output* axis. The two axes are orthogonal and, together, reach intermediate states that the tested loads, alone, would not reveal. The project’s incident log gathers seven episodes in six weeks, all with the same pattern: the primitives’ tests passed, integration failed silently, and the contract of a specification or an invariant detected the deviation. The window is short, there is no control group, and the process mixed the author’s work with automated analysis tools; these limitations are stated here. Still, the pattern suggests that, in phases of aggressive refactoring, keeping the formal invariants as a layer separate from the empirical tests shortens the time to detection. The specification amendments and the post-incident report are versioned in the v0.20.0-pre pre-release of Relativist.

6.3 Future Work

The work leaves four natural fronts of continuation, grouped by the nature of the extension each represents: experimental, empirical, encoding, and architectural. Each connects directly to a limit stated in Section 6.1.

Experimental front — validation on a physical LAN. The step that closes the proposed cycle is to run the distributed battery on a local network, with at least two heterogeneous physical machines. Swapping localhost for a LAN replaces the loopback latency, of a few microseconds, with real network latency, of hundreds of microseconds to milliseconds, which shifts the break-even point (Section 5.5) into the range where distributing the heaviest loads becomes advantageous. The execution requires operational care and is detailed in the v0.20.0-pre release of Relativist: synchronizing clocks via NTP, opening the TCP firewall rules, and comparing the *bincode* and *rkyv* serializations in zero-copy mode.

Empirical front — generalization of the sweep. The geometric sweep of Section 5.6 covered only one of the three planned loads, the CON-CON annihilation. The other two are the CON-DUP commutation and the DUP-DUP expansion. Extending it to them and raising the ceiling of N to 10^9 (in an environment with 128 GB of RAM or more)

would reveal at what scale each class of interaction rule begins to dominate the total cost, refining the decomposition presented in Section 5.

Encoding front — generalization of the polynomial encoder. The Horner encoder used in the ten demonstrations of Section 5.7 operates in a narrow envelope: polynomials of degree up to two, non-negative integer coefficients, and intermediate values within the ceiling of the Church-numeral encoder ($\sim 10\,000$). Widening it requires three steps. The first is to free the degree. The net read-back module needs to recognize composite duplication trees, of high fan-out. The second is to accept negative and rational coefficients, which requires going beyond Church numerals, for example with Church-encoded pairs representing relative integers or fractions. The third is to raise the magnitude ceiling above 10^4 , which requires a more compact numeric representation than that of unary Church numerals, since these make the number of agents grow linearly. Encoders for other domains, such as multipoint polynomial evaluation, matrix multiplication, and recurrences, would give rise to loads distributed by construction, with embedded data parallelism; they would be more favorable to parallelism than the synthetic loads of the main campaigns or the Horner demonstration itself, which is sequential by nature.

Architectural front — interfaces and robustness. Four natural continuations of the implementation remain. The first concerns dynamic worker admission, delivered only in detection mode. Completing the full path of reincorporating a worker that has left remains future work, already with a detailed plan; it reconstructs the retained state from the deltas already received. The second replaces the current round-robin partitioning with a strategy guided by the net’s topology, which minimizes the boundary wires and, with that, the traffic between workers. The third is a desktop graphical interface, along the lines of Docker Desktop and possibly over Tauri, which would open the system to users outside the command line, broadening the audience that can reproduce the experiments. The fourth is to formally mechanize the P1–P6 argument in an interactive proof assistant (Coq or Lean), which would raise the evidence from “verifiable by tests” to “verifiable by proof”, closing the cycle between Lafont’s theory and the empirical validation of this work.

REFERENCES

- [1] I. Foster, C. Kesselman, and S. Tuecke, “The anatomy of the grid: Enabling scalable virtual organizations,” *International Journal of High Performance Computing Applications*, vol. 15, no. 3, pp. 200–222, 2001.
- [2] H. Casanova, “Distributed computing research issues in grid computing,” *ACM SIGACT News*, vol. 33, no. 3, pp. 50–70, 2002, distributed Computing Column 8, edited by Sergio Rajsbbaum.
- [3] Y. Lafont, “Interaction combinators,” *Information and Computation*, vol. 137, no. 1, pp. 69–101, 1997, article No. IC972643.
- [4] —, “Interaction nets,” in *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’90)*. New York, NY, USA: ACM, 1990, pp. 95–108.
- [5] V. Taelin, “HVM2: A parallel evaluator for interaction combinators,” HigherOrderCO, Tech. Rep., 2024, work in progress. Disponivel em <https://github.com/HigherOrderCO/HVM>.
- [6] I. Mackie, “Static analysis of interaction nets for distributed implementations,” in *Proceedings of the LOMAPS Workshop on Analysis and Verification of Multiple-Agent Languages*, ser. Lecture Notes in Computer Science. Stockholm, Sweden: Springer-Verlag, 1997, pp. 217–231.

- [7] I. Mackie and J. S. Pinto, "Encoding linear logic with interaction combinators," *Information and Computation*, vol. 176, no. 2, pp. 153–186, 2002.
- [8] D. Mazza and N. J. Ross, "Full abstraction for set-based models of the symmetric interaction combinators," in *Foundations of Software Science and Computational Structures (FOSSACS 2012)*, ser. Lecture Notes in Computer Science, L. Birkedal, Ed., vol. 7213. Berlin, Heidelberg: Springer-Verlag, 2012, pp. 316–330.
- [9] W. Kahl, "A simple parallel implementation of interaction nets in Haskell," in *Developments in Computational Models 2014 (DCM 2014)*, ser. Electronic Proceedings in Theoretical Computer Science, U. Dal Lago and R. Harmer, Eds., vol. 179. Open Publishing Association, 2015, pp. 33–47.
- [10] J. S. Pinto, "Sequential and concurrent abstract machines for interaction nets," in *Proceedings of Foundations of Software Science and Computation Structures (FOSSACS 2000)*, ser. Lecture Notes in Computer Science, J. Tiuryn, Ed., vol. 1784. Springer-Verlag, 2000, pp. 267–282, on leave from Universidade do Minho.
- [11] I. Mackie and S. Sato, "Parallel evaluation of interaction nets: Some observations and examples," 2015, work-in-progress paper.
- [12] Y. Wang, Z. Luan, D. Qian, Y. Huang, T. Chen, B. Han, Y. Ren, K. Yu, and H. Jiang, "DDGrid: A grid computing environment with massive concurrency and fault-tolerance support," in *2008 Seventh International Conference on Grid and Cooperative Computing*. IEEE, 2008, pp. 5–14.
- [13] P. Arrighi, M. Costes, G. Dowek, and L. Maignan, "Space-time deterministic graph rewriting," *arXiv preprint arXiv:2404.05838v2*, October 2024, submitted to LIPICs. Université Paris-Saclay, Inria, CNRS, LMF / Univ Paris Est Creteil, LACL.